**EQUIPO:**

**Héctor Ulises Cacho González**

**Ciro Julián Cervantes Zamora**

**Dana Lizbeth Castañeda Sánchez**

**Concepcion Guadalupe Paniagua Gonzalez**

**Oscar Yael Hernández Rodríguez**

**Docente:**

**José Miguel Carrera Pacheco**

**Materia:**

**Desarrollo Web Profesional**

**Actividad:**

**S - 5. DOM y componentes dinámicos accesibles**

**FECHA:**

**8 de febrero de 2026**

# ANÁLISIS DE IMPACTO: API VS UI DINÁMICA

**Rol:** Backend Developer

**Objetivo:** Garantizar que las respuestas del servidor soporten los estados dinámicos del Frontend sin romper la accesibilidad.

## ESTRATEGIA DE GESTIÓN DE LATENCIA (LOADING STATE)

El Frontend implementará *Skeleton Loaders* (pantallas de carga esqueléticas) para mejorar la percepción de velocidad. Para soportar esto, el Backend debe garantizar lo siguiente:

### 1. Simulación de Latencia en Desarrollo

Los entornos locales (localhost) son irreales (0ms de latencia). Esto impide ver los estados de carga.

- **Acción Backend:** Se implementará un *Middleware de Latencia Artificial* configurable en el archivo .env.
    - API_DELAY=500 (Agrega 500ms de retraso a todas las respuestas en modo DEV).
    - **Objetivo:** Permitir al Frontend validar visualmente sus componentes isLoading.

### 2. Prevención de "Race Conditions" (Condiciones de Carrera)

En componentes de búsqueda dinámica (ej. el usuario escribe "L-E-C-H-E"), el Frontend enviará múltiples peticiones rápidas.

- **Riesgo:** Que la petición de "LE" llegue *después* de la de "LECHE", mostrando resultados incorrectos.
- **Solución Backend:** Todas las respuestas de búsqueda incluirán el parámetro de consulta original en el cuerpo de la respuesta (meta).
    - *JSON:* { "meta": { "query": "le" }, "data": [...] }
    - Esto permite al Frontend descartar respuestas que ya no coinciden con lo que el usuario está viendo en el input.

## PROTOCOLO DE ESTADOS VACÍOS (EMPTY STATE)

Uno de los errores más comunes que rompen aplicaciones React es el manejo inconsistente de datos nulos.

### 1. Regla de Oro: "Null Object Pattern" en Listas

El Frontend utiliza métodos como .map() o .filter() para renderizar listas. Si el Backend envía null, la aplicación se rompe (pantalla blanca de la muerte).

- **Contrato Estricto:**
  - Si una búsqueda no tiene resultados, el Backend devolverá **HTTP 200 OK** y un array vacío [].
  - **Prohibido:** Devolver HTTP 404 Not Found para búsquedas vacías. El 404 se reserva exclusivamente para rutas inexistentes o recursos específicos por ID que no se encuentran.

### 2. Diferenciación Semántica

Debemos ayudar al Frontend a decidir qué mensaje mostrar:

- **Escenario A (No hay resultados):** results: [] -> Frontend muestra: *"No encontramos productos con ese nombre"*.
- **Escenario B (Categoría vacía):** results: [] + category_exists: true -> Frontend muestra: *"Aún no hay productos en esta categoría, vuelve pronto"*.

## ESTABILIDAD DEL DOM VIRTUAL (REACT KEYS)

El Frontend tiene la tarea (FE-01) de investigar el DOM Virtual. React necesita una "Key" (clave única) para renderizar listas eficientemente y no perder el foco del teclado.

### 1. Garantía de IDs Únicos

El Backend garantiza que **cada objeto** retornado en una lista tendrá un id único, inmutable y persistente (UUID o Primary Key de PostgreSQL).

- **Riesgo:** Si usamos índices de array (0, 1, 2) como keys, y la lista cambia de orden, los lectores de pantalla (Accesibilidad) se confunden.
- **Compromiso BE:** Nunca enviar listas de objetos sin un identificador único.
  - *Malo:* ["Leche", "Huevo"]

- *Bueno:* [{ id: 101, name: "Leche" }, { id: 102, name: "Huevo" }]

## IMPACTO EN ACCESIBILIDAD (A11Y & ARIA)

La accesibilidad no es solo responsabilidad del Frontend. El Backend provee el "texto" que los lectores de pantalla leerán.

### 1. Mensajes de Error Humanizados (userMessage)

Los errores técnicos (Foreign Key Constraint Violation) son inútiles para el usuario final.

- **Estrategia:** El middleware de errores del Backend interceptará los errores técnicos y mapeará un mensaje amigable en un campo específico userMessage.

| Código Técnico | Situación | userMessage (Backend envía esto) | Acción ARIA (Frontend) |
|---|---|---|---|
| DB_TIMEOUT | Base de datos lenta | "El servidor está tardando mucho. Por favor, reintenta." | role="alert" |
| AUTH_FAIL | Password incorrecto | "Credenciales incorrectas. Verifica tu contraseña." | aria-invalid="true" |
| LIMIT_REACHED | Muchos intentos | "Demasiados intentos. Espera 1 minuto." | role="status" |

### 2. Soporte para Feedback Dinámico (aria-live)

Cuando el usuario guarda un producto en favoritos, el Backend debe responder rápido para que el Frontend pueda anunciar "Producto guardado" al lector de pantalla.

- **Optimización:** Los endpoints de "acción" (POST/PUT) serán optimizados para responder en <200ms, priorizando la respuesta UI sobre procesos secundarios (como enviar emails, que irán a una cola background).

## ANÁLISIS POR COMPONENTE CRÍTICO (RAPITI)

Desglose de cómo el Backend servirá a los componentes específicos de la UI de Rapiti:

**A. Buscador Predictivo (Typeahead)**

- **Desafío:** El usuario espera sugerencias mientras escribe.
- **Solución BE:** Endpoint /api/productos/search?q=XYZ&limit=5.
- **Optimización:** Uso de índices Trigram en PostgreSQL para búsquedas difusas (ej. usuario escribe "Lche" y el Backend encuentra "Leche").

**B. Mapa Interactivo de Tiendas**

- **Desafío:** Renderizar 500 pines en el mapa puede congelar el navegador.
- **Solución BE:** Implementación de **GeoJSON** ligero.
    - Solo se enviarán lat, lng, id y nombre en la vista de mapa.
    - Los detalles completos se cargarán solo al hacer clic en un pin (Lazy Loading).

**C. Modal de "Tienda Cerrada"**

- **Desafío:** Calcular si una tienda está abierta implica lógica compleja de fechas y zonas horarias.
- **Decisión Arquitectónica:** El Backend ("La fuente de la verdad") hará el cálculo.
    - Respuesta API: { "isOpen": false, "nextOpenTime": "Mañana 09:00 AM" }.
    - Esto evita que el Frontend tenga errores de lógica en JavaScript del cliente.

**MATRIZ DE PRUEBAS CRUZADAS (QA-BE-FE)**

Para garantizar la robustez, solicitamos a QA (Tarea QA-01) validar los siguientes escenarios extremos en nuestra API:

1. **Prueba de Estrés de Lista:** Solicitar una categoría con 1000 productos.
    - *Esperado:* Backend debe paginar (page 1 of 50). Frontend debe mostrar scroll infinito o paginación.
2. **Prueba de Inyección de HTML:** Enviar nombres de producto con scripts <script>alert(1)</script>.
    - *Esperado:* Backend debe sanitizar o Frontend debe escapar el contenido para evitar XSS.

3. **Prueba de "Network Failure":** Simular desconexión a mitad de una petición.
    - *Esperado:* Backend no se cuelga. Frontend muestra Toast de "Sin conexión"